

08_codegraph_opendesign

CodeGraph + OpenDesign as CORE Under-the-Hood Providers

Scope: Wire codegraph (§11.4.78–80) and opendesign (§11.4.162) as first-class, portable, under-the-hood providers across **HelixCode** → **HelixAgent** → **HelixLLM** → **LLMsVerifier**, so end-user productivity exceeds industry norms (Gemini CLI / Claude Code).

Date: 2026-07-06 **Author:** T1 deep-research + design subagent **Classification:** research + design (no source mutated in this pass)

Governance: §11.4.6 (facts, not guesses — every “current state” claim below is a probe result), §11.4.28 / CONST-045 (no hardcoded host paths), §11.4.78–80 (CodeGraph), §11.4.99 (latest-source verification — see the Sources footer), §11.4.162 (OpenDesign).

1. Current State (probed 2026-07-06 — §11.4.6)

1.1 CodeGraph

Fact	Value (probed)
<code>.mcp.json codegraph.command</code>	<code>/Users/milosvasic/.local/bin/codegraph</code> — macOS-absolute, broken on this Linux host
<code>.mcp.json codegraph.args</code>	<code>["serve", "--mcp", "--path", "/Volumes/T7/Projects/helix_code"]</code> — macOS /Volumes path, broken
Binary actually on PATH	<code>/home/milos/.local/bin/codegraph</code>
PATH binary <code>--version</code>	1.1.1
<code>npm global @colbymchenry/codegraph</code>	1.2.0 (drift — PATH binary is NOT the npm-global build; §11.4.80 update lever)
<code>npm latest published</code>	1.2.0 (published ~2026-07-05)

Fact	Value (probed)
.codegraph/config.json	present, version:1, broad include, large exclude
.codegraph/codegraph.db	present, 6.09 GB ; index = 102,708 files / 1.78M nodes / 6.75M edges
node / npm	node v24.18.0, npm 11.16.0

Consequences: (a) the .mcp.json entry cannot launch on Linux (path rot — the exact CONST-045 / §11.4.28 hardcoded-host-path defect); (b) a **version skew** — the launched binary would be 1.1.1 while the org-mandated latest is 1.2.0; (c) the index is **bloated** — 102k files / 6 GB is far more than HelixCode + own-org submodules; it is pulling in submodules/helix_qa/tools/opensource/** (appium, unstructured, docling, midscene, marker, ui-tars ...) — **third-party vendored trees that §11.4.79 says must be EXCLUDED**, not indexed.

.codegraph/config.json **exclude** already drops third-party at the repo root (dependencies/LLama_CPP, dependencies/Ollama, dependencies/HuggingFace_Hub, cli_agents/**, cli_agents_resources/**, github_pages_website/**, awesome-ai-memory/**) — but it does **not** exclude the vendored subtree inside submodules/helix_qa/tools/opensource/**, nor dependencies/HelixDevelopment/helix_llm/** (a duplicate of submodules/helix_llm). Own-org submodules (submodules/helix_agent, submodules/helix_llm, submodules/llms_verifier, submodules/llm_orchestrator, submodules/llm_provider, submodules/vision_engine, submodules/doc_processor, submodules/challenges, submodules/security, submodules/containers, submodules/helix_qa **source**) are **not** excluded → they ARE indexed (satisfies §11.4.79 (a)); the gap is that vendored third-party under them is ALSO indexed (violates §11.4.79 (b) and bloats the DB).

Reference scripts already present (consumed **by reference**, §11.4.80 — do not copy): constitution/scripts/codegraph_update.sh, codegraph_sync.sh, codegraph_update_and_resync.sh; local scripts/codegraph_setup.sh + codegraph_validate.sh; docs/codegraph/Status.md + Status_Summary.md (+ html/pdf) ledgers exist.

1.2 OpenDesign

Fact	Value (probed)
.mcp.json open-design	disabled: true
open-design.command	npx -y open-design-mcp
open-design.env	OD_DAEMON_URL=http://localhost:7456

Fact	Value (probed)
npm open-design-mcp latest	0.16.1 (the MCP shim package — resolvable)
OD daemon on :7456	NOT running (curl http://localhost:7456/api/health empty)
od on PATH	/usr/bin/od = GNU coreutils octal-dump — NAME COLLISION ; the OpenDesign od CLI is NOT installed
OpenDesign usage in HelixCode source	none (greenfield — only the disabled MCP entry)
od MCP tools available in-session (deferred)	od_list_projects, od_get_project, od_create_project, od_update_project, od_delete_project, od_compose_brief, od_generate_design, od_lint_artifact, od_save_artifact, od_save_project_file

Consequences: (a) OpenDesign is fully unwired — disabled MCP + no daemon + no source consumption; (b) the **od command name collides with coreutils** on Linux, so the constitution’s “od mcp install” one-liner is unusable as-is here — OpenDesign must be driven via the **daemon + npx open-design-mcp MCP shim** (or an explicit non-od binary path), never bare od; (c) enabling it requires the OD daemon up on :7456 with BYOK env.

1.3 HelixCode / HelixAgent / HelixLLM integration seams (probed)

- **HelixCode** helix_code/internal/: repomap (Aider-style repo map / file ranker for prompt context), context, memory, projectmemory, cognee, rules, mcp (an MCP **client** — lifecycle/config/transport), provider + providers (LLM providers), editor, theme (5 semantic roles × ANSI16/256/truecolor — the **TUI token surface**).
- **HelixCode UI surfaces** helix_code/applications/: desktop (Fyne), terminal_ui (tview/tcell), plus ios/android/aurora_os/harmony_os; internal/theme/{types,loader,builtin,detect}.go is the theme engine (light/dark auto-detect + loader).

- **HelixAgent** submodules/helix_agent/internal/: rag (hybrid, hyde, advanced pipeline), mcp, memory, storage — the retrieval/agent-context layer.
 - **HelixLLM** submodules/helix_llm/: provider/serving layer (cmd, internal, pkg); no RAG/embedding package — it serves models; code-context is assembled **upstream** in HelixCode/HelixAgent, not inside HelixLLM.
 - **LLMsVerifier** submodules/llms_verifier: the CONST-036 single source of truth for model/provider/capability metadata (unchanged by this work; it is a consumer of the same MCP wiring, not a code-context producer).
-

2. CodeGraph — latest interface (verified 2026-07-06, §11.4.99)

Ground truth from codegraph --help on this host + the upstream README:

- **Start MCP server:** codegraph serve --mcp (stdio). -p/--path is **optional in MCP mode** — it uses **rootUri** from the client; --no-watch disables the file watcher (useful on slow FS).

- **Portable .mcp.json (upstream-recommended):**

```
{ "mcpServers": { "codegraph": { "type": "stdio",
  "command": "codegraph", "args": ["serve","--mcp"] } } }
```

command: "codegraph" (bare, resolved on PATH) — **no host path, no --path** → satisfies §11.4.28 / CONST-045.

- **MCP tools:** primary codegraph_explore (“how does X work / how does X reach Y / survey an area” in one call); unlisted-by-default codegraph_node, codegraph_search, codegraph_callers, codegraph_callees, codegraph_impact, codegraph_files, codegraph_status — re-enable via env CODEGRAPH_MCP_TOOLS=explore,node,search,callers,....
- **Permissions:** auto-allow with "permissions": { "allow": ["mcp__codegraph__*"] } in .claude/settings.json.
- **CLI parity (for scripts/agents):** codegraph explore|node|query|callers|callees|impact|affected|files|status|sync|index — same output as the MCP tools, callable from bash.
- **Config:** .codegraph/config.json (present) — include / exclude (gitignore-style, repo-root-relative), maxFileSize (default 1 MiB), extractDocstrings, trackCallSites, extensions (map custom ext →

language). `codegraph install` writes the MCP entry into Claude Code / Cursor / Codex / opencode / Hermes automatically (no hand-editing needed).

- **Lifecycle:** `init` (first index), `sync` (incremental), `index` (full rebuild), `upgrade` (self-update). `.codegraph/codegraph.db` is the SQLite graph — gitignored, with `codegraph index` as its §11.4.77 regeneration mechanism.

3. OpenDesign — latest interface (verified 2026-07-06, §11.4.99)

- **Architecture:** a local-first desktop app + **daemon on :7456** (web UI at `http://localhost:7456`) + an **MCP server** (`open-design-mcp`, npm 0.16.1). Your coding agent is the design engine (BYOK); artifacts export to HTML/PDF/PPTX/MP4 as real files.
 - **Design systems / tokens:** portable Markdown `DESIGN.md` (9-section schema — Color, Typography, Spacing, Layout, Components, Motion, Voice, Brand, Anti-patterns) under `design-systems/<brand>/`, each with **light + dark** variants and a generated `tokens.css`. “Switch a system → the next render uses the new tokens.” This is the token/theme source of truth §11.4.162 mandates.
 - **MCP tools (this session’s schemas):** `od_create_project / od_get_project / od_update_project / od_list_projects / od_delete_project` (project + stored customInstructions / brand rules), `od_compose_brief`, `od_generate_design` (BYOK render — needs `BYOK_BASE_URL + BYOK_API_KEY + BYOK_MODEL`, `kind ∈ {prototype, deck, template, image, video, audio}`, `maxTokens` up to 200k), `od_lint_artifact` (design-lint a rendered artifact), `od_save_artifact / od_save_project_file`.
 - **BYOK proxy:** `POST /api/proxy/` {`anthropic, openai, azure, google, ollama, senseaudio`} / `stream`. Env: `OD_DAEMON_URL`, `BYOK_BASE_URL`, `BYOK_API_KEY`, `BYOK_MODEL`, `OD_BIND_HOST` (default 127.0.0.1), `OD_ALLOWED_ORIGINS`, `OD_ALLOWED_INTERNAL_HOSTS` (SSRF opt-out for a local Ollama/HelixLLM base URL).
 - **CLI caveat (Linux):** the OpenDesign CLI is `od`, which **collides with coreutils /usr/bin/od**. Do **not** rely on bare `od`; drive OpenDesign through the `daemon + npx open-design-mcp MCP shim`, and if a CLI is needed install it under an unambiguous name (e.g. `opendesign`) or an explicit path — never shadow `coreutils`.
-

4. Portable Wiring Fix (Linux-correct, host-agnostic)

4.1 .mcp.json — replace both entries

```
{
  "mcpServers": {
    "codegraph": {
      "type": "stdio",
      "command": "codegraph",
      "args": ["serve", "--mcp"],
      "env": { "CODEGRAPH_MCP_TOOLS":
        "explore,node,search,callers,callees,impact,files,status" }
    },
    "open-design": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "open-design-mcp"],
      "env": {
        "OD_DAEMON_URL": "http://localhost:7456",
        "BYOK_BASE_URL": "${HELIX_OD_BYOK_BASE_URL}",
        "BYOK_API_KEY": "${HELIX_OD_BYOK_API_KEY}",
        "BYOK_MODEL": "${HELIX_OD_BYOK_MODEL}"
      },
      "disabled": false,
      "autoApprove": []
    },
    "media-validator": { "...unchanged..." }
  }
}
```

Key changes (each maps to a probed defect): 1. **codegraph** → bare codegraph on PATH, drop the macOS binary path AND the /Volumes --path (rootUri supplies it). CONST-045 / §11.4.28 satisfied; portable to macOS/Linux/WSL. Add CODEGRAPH_MCP_TOOLS so impact/callers/callees (the high-value graph tools) are exposed, not just explore. 2. **open-design** → disabled:false; BYOK keys come from **env expansion of .env secrets** (HELIX_OD_BYOK_*, mode-0600, gitignored — §11.4.10 / CONST-042), never inline. Point BYOK_BASE_URL at a HelixLLM/Ollama base for a fully-local design engine (add its host to OD_ALLOWED_INTERNAL_HOSTS). 3. .claude/settings.json → "permissions": { "allow": ["mcp__codegraph__*", "mcp__open-design__*"] } for zero-friction use.

4.2 CodeGraph index scope — enforce §11.4.79 (own-org IN, third-party OUT)

Add to `.codegraph/config.json` exclude (keeps own-org **source** indexed, drops vendored third-party bloat that inflated the DB to 6 GB / 102k files):

```
"submodules/helix_qa/tools/opensource/**",  
"submodules/**/vendor/**",  
"submodules/**/node_modules/**",  
"dependencies/HelixDevelopment/**"           // duplicate of submodules/helix
```

Leave `submodules/helix_agent/**`, `submodules/helix_llm/**`, `submodules/llms_verifier/**`, `submodules/llm_orchestrator/**`, `submodules/llm_provider/**`, `submodules/vision_engine/**`, `submodules/doc_processor/**`, `submodules/security/**`, `submodules/containers/**` **indexed** (own-org, §11.4.79 (a)). Record the include/exclude rationale in a `.codegraph/config.json` comment sidecar (§11.4.79 audit trail). Then `bash scripts/codegraph_setup.sh` (re-index) + `bash scripts/codegraph_validate.sh` with a probe resolving a symbol that lives **ONLY** inside an own-org submodule (the §11.4.79 (4) cross-submodule proof) + the §1.1 paired mutation (temporarily exclude that submodule → validate **MUST FAIL** → restore).

4.3 Version drift + weekly automation (§11.4.80)

- Reconcile the PATH binary (1.1.1) with npm-global (1.2.0): run `constitution/scripts/codegraph_update.sh` (npm-installs latest, anti-bluff verifies codegraph --version reflects it) then `codegraph_sync.sh`. Ensure the npm-global bin dir precedes `/home/milos/.local/bin` on PATH, or repoint `/home/milos/.local/bin/codegraph`.
- Schedule `codegraph_update_and_resync.sh` weekly (§11.4.80 cadence floor); append results to `docs/codegraph/Status.md` + export html/pdf via the §11.4.65 exporter.

4.4 OpenDesign bring-up (one-time, outside test execution — §11.4.98(B))

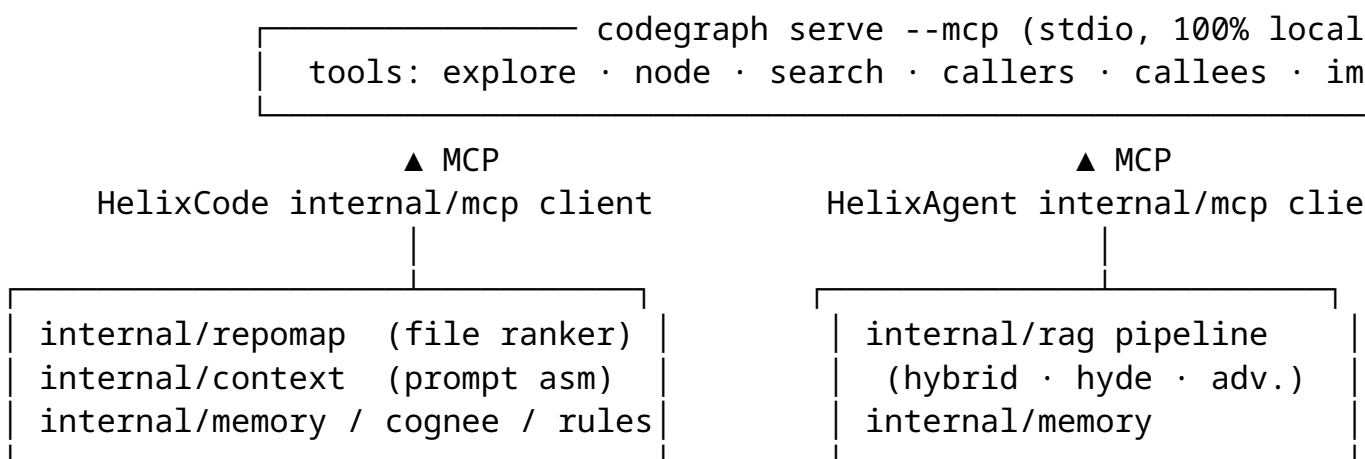
1. Install the OpenDesign desktop/daemon (per upstream) — start daemon on `:7456` (`OD_BIND_HOST=127.0.0.1`). Health-gate: `curl -fsS localhost:7456/api/health`.
2. Put `HELIX_OD_BYOK_*` in `.env` (0600, gitignored); add HelixLLM/Ollama host to `OD_ALLOWED_INTERNAL_HOSTS` if BYOK points local.
3. `od_create_project` a **helixcode-brand** project seeded with HelixCode brand assets (assets/ logos/themes) + `customInstructions` = the HelixCode design rules → becomes the stored brand contract every `od_generate_design` merges.

4. Add a §11.4.78-style unforgeable challenge: an agent fact obtainable only via an `od_*` MCP call (e.g. `od_list_projects` returns `helixcode-brand`) + a codegraph fact (`codegraph_status` node count) → proves both providers are genuinely wired, not faked.

5. Core-Provider Integration Design

Both tools become **under-the-hood providers** (services the platform calls transparently), not operator-facing add-ons. Two clean seams already exist: HelixCode/HelixAgent each ship an **MCP client** (`internal/mcp`), and HelixCode ships a **repo-map/context** layer and a **theme** engine. We plug the two MCP servers into those.

5.1 CodeGraph = the code-intelligence provider (RAG + agent context + retrieval)



Concrete seams:

1. **internal/repomap → CodeGraph-backed ranking.** Today `repomap` builds an Aider-style map from tree-sitter + a heuristic `file_ranker`. Add a `CodeGraphSource` that, given the current task/query, calls `codegraph_explore / codegraph_node` (via `internal/mcp`) and returns graph-ranked symbols + call trails. This is the single biggest productivity lever: the model gets a **precise dependency-aware slice** instead of a whole-file dump — the documented CodeGraph “90% fewer exploration tokens / tool calls” effect. Keep the tree-sitter path as the offline fallback when the MCP server is down (§11.4.6 honest degradation).
2. **internal/context / cognee / memory → CodeGraph as a retrieval source.** When assembling an LLM prompt, add codegraph results as a first-class context contributor (symbol source + callers/callees), deduped against memory/RAG. Cache by symbol so repeated turns don’t re-query.

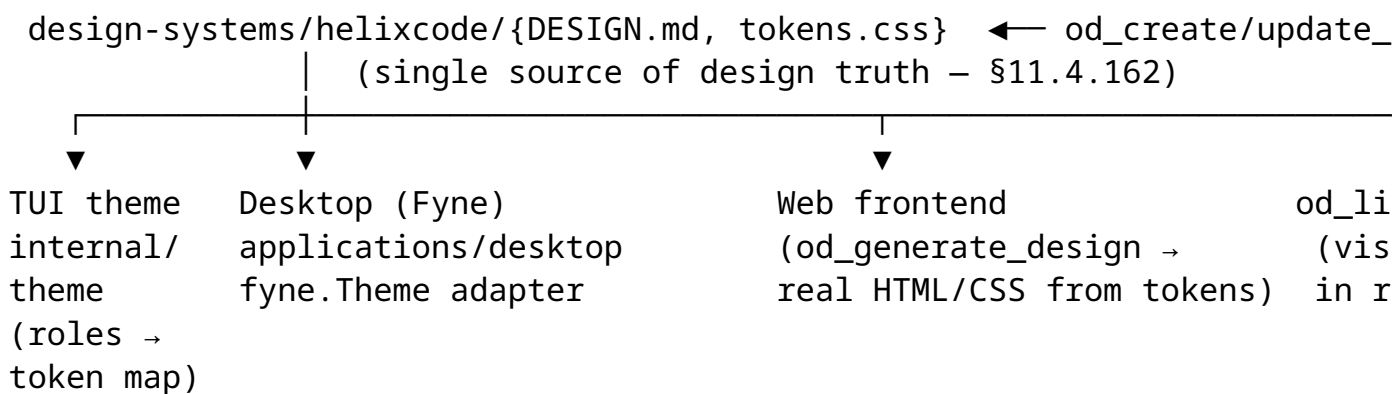
3. **HelixAgent internal/rag → CodeGraph as a retriever alongside the vector store.** The hybrid retriever already fuses sources; register a codegraph retriever whose “hits” are graph nodes/edges. hyde (hypothetical-doc) queries can be grounded by first resolving the real symbols via codegraph_search, cutting hallucinated file paths.
4. **Impact-aware agent actions.** Before an edit, agents call codegraph_impact <symbol> / codegraph affected <files> to enumerate blast radius (§11.4.145 impact-research angle 2, §11.4.108 runtime-signature scoping) and to pick the tests to run — turning “run the whole suite” into “run the affected suite” (§11.4.82 iteration-speedup).
5. **LLMsVerifier / HelixLLM.** HelixLLM stays a pure serving layer; it does **not** import codegraph (code-context is assembled upstream). LLMsVerifier remains the model-metadata SoT and simply shares the same MCP wiring. No coupling added — preserves §11.4.28 decoupling.

Provider contract (Go interface sketch, HelixCode side — lives in internal/provider or a new internal/codeintel):

```
type CodeIntel interface {
  Explore(ctx, query) ([]Symbol, error);
  Node(ctx, name) (Symbol, error);
  Impact(ctx, symbol) ([]Ref, error);
  Affected(ctx, files) ([]TestFile, error) };
```

with an mcpCodeIntel impl over internal/mcp and a nullCodeIntel fallback. Decoupled: the submodules never hardcode a codegraph path — they consume the interface / the MCP endpoint injected by config (§11.4.28 / CONST-045).

5.2 OpenDesign = the design-system provider (all HelixCode UI tokens/themes)



Concrete seams:

1. **Token single-source.** Create design-systems/helixcode/ (via od_create_project + od_save_project_file) holding DESIGN.md (9-section, light+dark) and generated tokens.css. This is the §11.4.162 mandated token/theme source; brand colors come from assets/ (no ad-hoc CSS anywhere).

2. **TUI (internal/theme) adapter.** `internal/theme` maps 5 semantic roles (info/warn/ error/highlight/dim) × ANSI depths. Add a `loader.go` path that reads the `OpenDesign tokens.css` (semantic → hex) and down-samples to ANSI16/256/truecolor, so the terminal UI and the web/desktop UIs share one palette. `detect.go` already picks light/dark → select the matching `OpenDesign` variant.
3. **Desktop (Fyne) adapter.** Implement `fyne.Theme` backed by the `OpenDesign` tokens (Color/Font/Size lookups → token values), giving the Fyne GUI the same brand + light/dark.
4. **Web frontend generation.** Use `od_generate_design` (BYOK → HelixLLM/Ollama) to render HTML/CSS **from the tokens** rather than hand-CSS; `od_lint_artifact` becomes a design gate wired into the §11.4.162 / §11.4.125 review (no overlaps, no font collisions, WCAG-adjacent checks), each UI change carrying a visual-regression artifact.
5. **Decoupling (§11.4.28).** `OpenDesign` is consumed by reference (daemon + MCP); `HelixCode` never vendors it. The brand project is `HelixCode`-owned data; the engine stays project-agnostic. If a needed UI primitive is missing, extend `OpenDesign` upstream (§11.4.74 extend-don't-reimplement), never fork a local CSS hack.

5.3 Why this beats industry norms (productivity levers)

- **Fewer tokens / fewer tool calls per task.** `CodeGraph`-ranked context replaces read-the-whole-file exploration — the documented ~90% exploration-cost cut — directly serving §11.4.141 token-efficiency. Gemini/Claude Code re-grep and re-read; `HelixCode/Agent` query a pre-built local graph.
 - **Impact-scoped edits + tests.** `impact/affected` turn full-suite runs into affected-set runs (§11.4.82), and feed §11.4.145 blast-radius + §11.4.108 runtime-signature scoping — a correctness lever competitors lack out of the box.
 - **One brand, every surface, no drift.** `OpenDesign` tokens govern TUI + desktop + web from a single light/dark source with a machine `od_lint_artifact` gate — eliminating the per-surface CSS divergence that plagues multi-frontend products.
 - **100% local, zero-cloud, BYOK.** `CodeGraph SQLite` is local; `OpenDesign BYOK` can point at `HelixLLM/Ollama` — no data egress, no per-call SaaS cost, aligned with the offline mandates.
 - **Recursive reuse.** Both are org-wide providers (§11.4.79 indexes every own-org submodule; §11.4.162 governs every UI), so the leverage compounds across `HelixAgent/HelixLLM/LLMsVerifier` rather than being a `HelixCode`-only tweak.
-

6. Top Risks

1. **CodeGraph index bloat / staleness.** 6 GB / 102k files today because vendored third-party is indexed; without the \$4.2 excludes, re-index is slow and RAG hits are polluted by appium/docling/etc. Mitigation: the exclude patches + weekly sync (§11.4.80) + a size budget in `codegraph_validate.sh`. Also reconcile the 1.1.1 ↔ 1.2.0 drift before wiring.
 2. **OpenDesign operational fragility.** The daemon must be up on `:7456` with BYOK secrets, and the `od` ↔ `coreutils` name collision breaks the upstream one-liner on Linux. If the daemon is down, `od_generate_design` fails → UI generation blocked. Mitigation: daemon health-gate before use, honest §11.4.3 SKIP-with-reason when absent (never a faked design PASS), BYOK pointed at local HelixLLM/Ollama, drive only via `npx open-design-mcp` (never bare `od`).
 3. **Coupling / decoupling regressions (§11.4.28 / CONST-045).** Re-introducing a hardcoded host path (the original defect) or letting an own-org submodule reach into a codegraph/OD host path would break portability + reuse. Mitigation: bare-command MCP entries, env-injected BYOK, the interface-based CodeIntel seam, and a §11.4.109-class guard/gate that greps `.mcp.json` + submodule sources for absolute `/Users//Volumes//home/<user>` paths.
-

Sources verified 2026-07-06

- CodeGraph npm (latest 1.2.0): <https://www.npmjs.com/package/@colbymchenry/codegraph>
- CodeGraph repo + README (MCP tools, `serve --mcp`, `.mcp.json`, config schema): <https://github.com/colbymchenry/codegraph> · <https://github.com/colbymchenry/codegraph/blob/main/README.md>
- CodeGraph releases / changelog: <https://github.com/colbymchenry/codegraph/releases> · <https://github.com/colbymchenry/codegraph/blob/main/CHANGELOG.md>
- CodeGraph CLI ground truth: `codegraph --help` / `codegraph serve --help` / `codegraph status` on host `/home/milos/.local/bin/codegraph` (binary `--version` = 1.1.1; npm-global = 1.2.0), 2026-07-06.
- OpenDesign repo + README (daemon `:7456`, MCP server, DESIGN.md 9-section tokens, light/dark, BYOK proxy): <https://github.com/nexu-io/open-design> · <https://raw.githubusercontent.com/nexu-io/open-design/main/README.md>
- OpenDesign releases (v0.8.0 “Plugin Everything”): <https://github.com/nexu-io/open-design/releases>
- OpenDesign QUICKSTART: <https://github.com/nexu-io/open-design/blob/main/QUICKSTART.md>

- npm open-design-mcp (latest 0.16.1): npm view open-design-mcp version, 2026-07-06.
- OpenDesign od_* MCP tool schemas: this session's loaded tool definitions (od_generate_design, od_list_projects, od_create_project, od_update_project, od_lint_artifact, od_compose_brief, od_save_artifact, od_save_project_file, od_get_project, od_delete_project).
- Local probes (2026-07-06): .mcp.json, .codegraph/config.json, codegraph status (102,708 files / 1.78M nodes / 6.09 GB), curl localhost:7456 (daemon down), command -v od → /usr/bin/od (coreutils collision), .env HELIX_RELEASE_PREFIX=helixcode.